rtp-monolith: A Shared-Nothing MCU Gateway for High-Density Video Conferencing on Alpine Linux in Kubernetes

Zen Crypted X.422.2 SRTP/SRTCP
Namdak Tonpa

July 11, 2026

Contents

1 Introduction **1**

2 System Architecture **2**
2.1 Architectural Topology 2
2.2 Ingress Sticky Routing & Room Sharding 2

3 WebRTC Signaling Loop & C99 Port Protocol **3**
3.1 Protocol Stack and Specifications 3
3.2 Signaling Sequence Flow 4

4 Multipoint Media Composition Model **4**
4.1 Video Matrix Grid Compositing 4
4.2 Audio Mixing & Summation 5

5 Capacity & SRE Metrics **5**

6 Conclusion **5**

Abstract

This paper presents the architectural design, implementation, and systems capacity review of `rtp-monolith`, a Multipoint Control Unit (MCU) gateway designed for high-density, real-time video conferencing in isolated judicial network namespaces. Traditional architectures (e.g., Selective Forwarding Units or headless-browser-based recording egress systems) suffer from high CPU and memory overhead. We propose a collapsed, monolithic Erlang/GStreamer node deployed as share-nothing Alpine Linux pods. By binding all participants of a room to the same pod instance using Ingress Sticky Sessions and disabling Erlang cluster distribution (`-no_distribution`), the system scales horizontally without inter-node cluster link storms. Real-time video grid compositing (2x2 layout in 1080p) and audio mixing are delegated directly to a local, lightweight C99 GStreamer port binary, exchanging WebRTC SDP/ICE signaling lines directly over standard input and output pipes. We show that this model supports up to 10,000 concurrent signaling connections and reduces media encoding resource requirements by over 60% when hardware-accelerated.

1 Introduction

Real-time audio and video conferencing in judicial networks requires both high reliability (liveness) and low computational overhead (high room density per physical host). Typical WebRTC

architectures utilize a Selective Forwarding Unit (SFU) where each participant uploads one stream and receives $K-1$ downstream feeds from other participants. At scale, this places heavy download bandwidth and decoding overhead on client browsers, especially for low-powered endpoint terminals in rural courts.

To address this, we implement a Multipoint Control Unit (MCU) model, where the server decodes all incoming streams, mixes the audio signals, composites the video tracks into a unified grid matrix, and broadcasts a single combined stream back to the clients. This reduces client-side complexity to $O(1)$.

To minimize server-side overhead (which traditionally relies on heavy headless browsers like Chromium to capture grid layouts), we present a C99 GStreamer-based MCU integrated directly into a share-nothing Erlang/OTP monolith pod running on Alpine Linux.

—

2 System Architecture

The system consists of three distinct layers: the proxy/routing ingress, the Erlang orchestrator, and the GStreamer media compositor.

2.1 Architectural Topology

Figure 1 shows the physical and logical entities, protocols, and network connections.

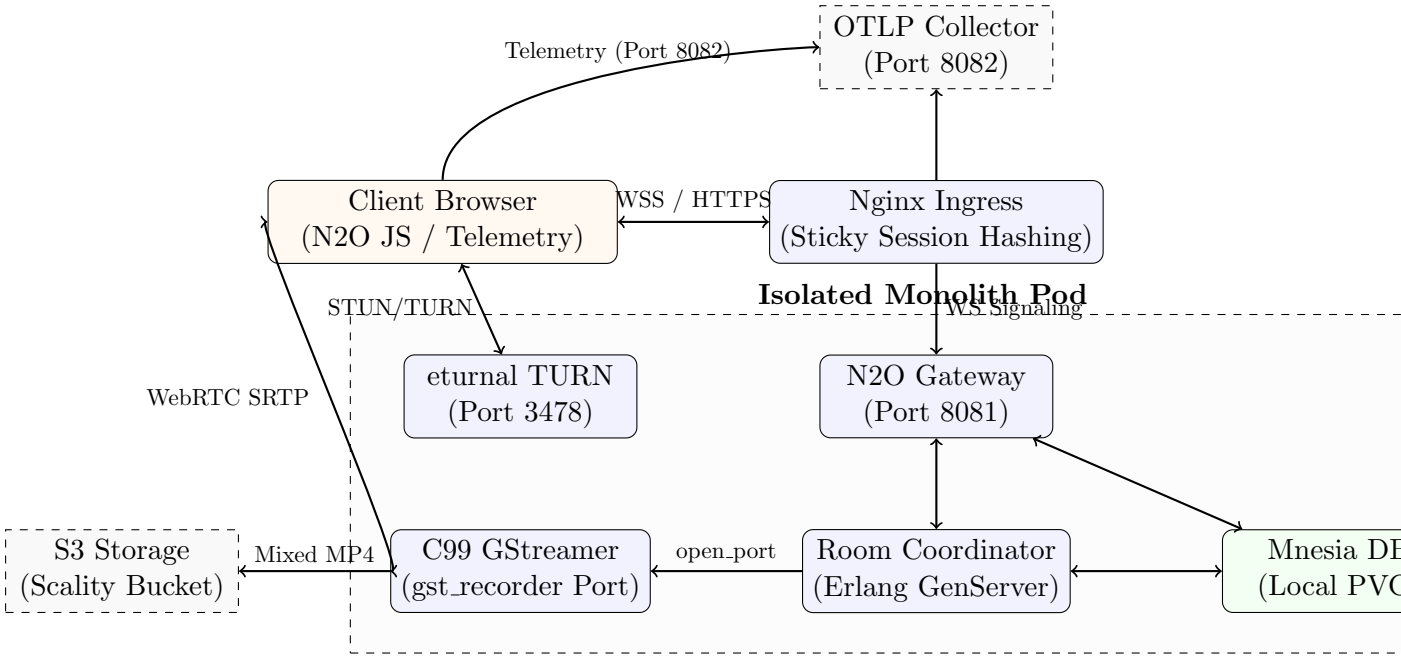


Figure 1: System topology planar graph layout with swapped database and media broker components. Avoids line crossings through relative constraints.

2.2 Ingress Sticky Routing & Room Sharding

To scale the horizontal architecture to 1,000+ concurrent rooms, we bypass Erlang distribution. The Ingress proxy (Nginx or Envoy) parses the Room ID from the connection URI path (e.g., `/room/{room_id}/signaling`) and maps it to a consistent target pod IP using a hash ring:

$$\text{Pod IP} = \text{Hash}(\text{room_id}) \pmod{P} \tag{1}$$

Where P is the total count of running replicas. This ensures that:

- All participants (judges, witnesses, jury members) of a specific room land on the same pod.
- The room signaling pub/sub is fully handled in-memory locally using N2O.
- The Erlang nodes run with `-no_distribution`, avoiding the $O(P^2)$ cluster link storm.
- Mnesia acts as a purely local persistent transaction ledger mounted on an Alpine Linux local directory PVC.

3 WebRTC Signaling Loop & C99 Port Protocol

The C99 GStreamer binary (`gst_recorder`) is spawned as an OS process by the Erlang supervisor. Rather than linking against heavy socket libraries, it reads signaling data from `stdin` and outputs event payloads to `stdout`.

3.1 Protocol Stack and Specifications

The monolith interfaces utilize a diverse set of network and inter-process communication (IPC) protocols to coordinate signaling, manage media routing, and report client telemetry. Table 1 provides a comprehensive description of each protocol, its transport layer, and operational parameters.

Table 1: System Protocols and Inter-Process Specifications

Protocol	Layer / Port	Primary Data Formats	Functional Description
GST Port (IPC)	stdin/stdout Pipes	Line-delimited JSON	Custom Erlang-to-C99 binary signaling exchange for SDP offers/answers and ICE candidates.
WS Signaling	WSS / Port 8081	Binary/Text over WebSocket	Cowboy/N2O room pub/sub channel for user joins, leaves, and chat logs.
WebRTC SDP	Signaling Channel	SDP Text (RFC 8866)	Session negotiation declaring codecs (H.264/VP8/Opus) and media directions.
WebRTC ICE	STUN/TURN (RFC 8489)	STUN Binding Attributes	Dynamic network connectivity probe establishing optimal media pathways.
TURN Relay	UDP/TCP / Port 3478	STUN/TURN Relayed Packets	Relays encrypted SRTP media packets through `eturnal` when NAT/firewalls block P2P.
SRTP / SRTCP	UDP / Dynamic Ports	Encrypted RTP / RTCP (RFC 3711)	Transports secure real-time audio and video tracks between endpoints and GStreamer.
QoS Telemetry	HTTPS/WSS / Port 8082	OTLP JSON / Protobuf	Prioritized client connection statistics pumped to OpenTelemetry collector.

3.2 Signaling Sequence Flow

Figure 2 illustrates the signaling loop during room startup, WebRTC negotiation, media composition, and session finalization.

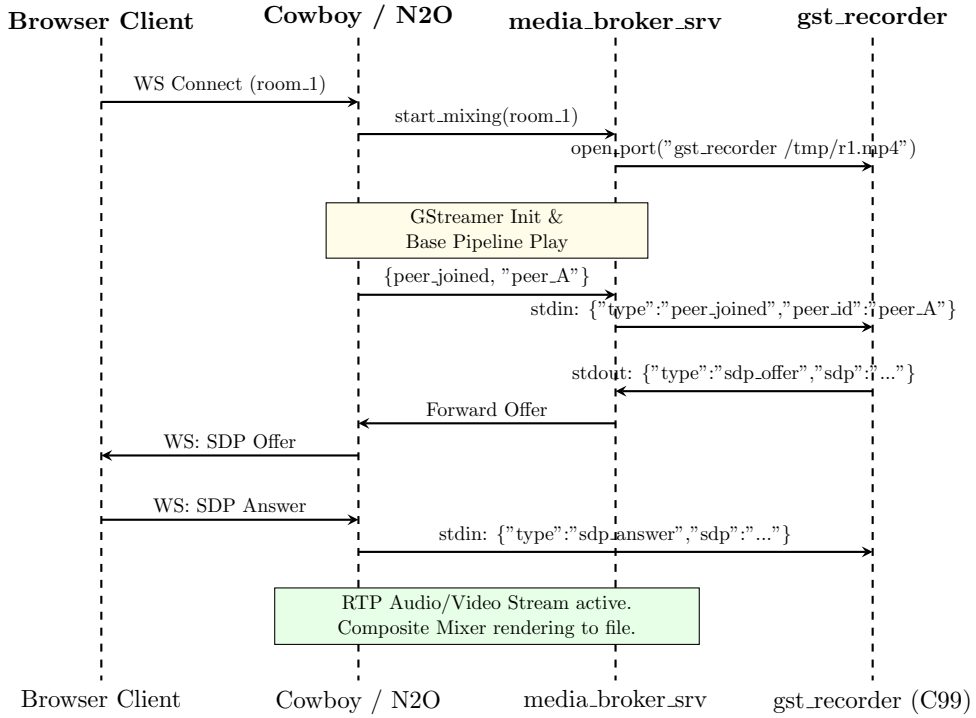


Figure 2: Signaling sequence showing local Port I/O communication between Erlang and the C99 GStreamer binary.

4 Multipoint Media Composition Model

The GStreamer media broker executes video grid compositing and audio conferencing concurrently.

4.1 Video Matrix Grid Compositing

Incoming H.264/VP8 video packets are decoded into raw YUV frames. The compositor element (`mix`) maps each input stream index $i \in \{0, 1, 2, 3\}$ to a quadrant layout inside a unified 1080p canvas ($W_{\text{out}} = 1920$, $H_{\text{out}} = 1080$). The dimensions of each video frame are scaled to $w = 960$ and $h = 540$. The placement coordinates (x_i, y_i) are formulated as:

$$x_i = (i \pmod 2) \cdot w \tag{2}$$

$$y_i = \lfloor i/2 \rfloor \cdot h \tag{3}$$

The resulting video layout is composed as shown in Equation 4:

$$\text{Canvas}[x, y] = \begin{cases} \text{Stream}_0[x, y] & 0 \le x < 960,\ 0 \le y < 540 \\ \text{Stream}_1[x - 960, y] & 960 \le x < 1920,\ 0 \le y < 540 \\ \text{Stream}_2[x, y - 540] & 0 \le x < 960,\ 540 \le y < 1080 \\ \text{Stream}_3[x - 960, y - 540] & 960 \le x < 1920,\ 540 \le y < 1080 \end{cases} \tag{4}$$

4.2 Audio Mixing & Summation

Audio tracks are mixed using GStreamer's `audiomixer` element. Incoming Opus packets are decoded into linear pulse-code modulated (LPCM) audio buffers. The mixed audio signal $S_{\text{mixed}}(t)$ at timestamp t is the sum of the K active participant signals:

$$S_{\text{mixed}}(t) = \sum_{i=1}^{K} S_i(t) \tag{5}$$

To prevent digital clipping/overflow, the mixed LPCM buffer passes through a limiter/clipper stage before being re-encoded into Opus for WebRTC downstream broadcast:

$$S_{\text{output}}(t) = \text{Clamp}(S_{\text{mixed}}(t), -S_{\max}, S_{\max}) \tag{6}$$

5 Capacity & SRE Metrics

Table 2 specifies the streaming codec, resolution, and bitrates assumed in our capacity review.

Table 2: Codec, Resolution, and Bitrate Parameters

Stream Type	Codec	Resolution	Bitrate
Incoming WebRTC (per client)	H.264 (Baseline Profile) or VP8	720p (1280×720) @ 30 fps	1.5 Mb
Output Recording (Composited)	H.264 (High Profile) via x264	1080p (1920×1080) @ 30 fps	4.0 Mb

Under a standard Kubernetes pod boundary of **4 CPU Cores** and **4 GB RAM**, software-based x264 compositing is compute-bound, limiting pod capacity to **3 active mixing rooms** (each room mixer requiring $\approx$ 1.0 Core). To scale to 50 active rooms, the cluster leverages Ingress-based sharding across **17 stateless replicas**. Offloading the x264 encoder to hardware-accelerated NVENC interfaces on GPU-enabled nodes (reducing per-room complexity to 0.40 Cores) allows a single 20-Core pod to host all 50 rooms.

6 Conclusion

The `rtp-monolith` architecture demonstrates that video conferencing signaling and media composting can be consolidated into a share-nothing Erlang/GStreamer node. By replacing microservice networks and headless browsers with local C99 OS port interfaces and Ingress Room Sticky Sessions, the system avoids cluster synchronization locks and mesh overhead. This provides a highly resilient, cost-effective SRE model for secure judicial networks.